Terraform and infrastructure as code

Describe infrastructure as code, use advanced HCL, manage state and secrets safely, and build real AWS and Azure stacks.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Infrastructure as code and HCL basics (14 min)
2. Providers, resources and data sources (15 min)
3. Variables, locals, outputs and validation (14 min)
4. Expressions, loops and dynamic blocks (16 min)
5. State, backends and locking (16 min)
6. Modules and project structure (15 min)
7. Secrets and sensitive data in Terraform (14 min)
8. End to end: a web stack on AWS and on Azure (18 min)
9. Team workflows, drift and testing (15 min)
10. Testing Terraform code (14 min)
11. Environments, repository layout and team workflows (14 min)
12. Importing resources and refactoring safely (13 min)
13. Large configurations: structure, speed and blast radius (12 min)
14. Debugging Terraform plans, state and providers (15 min)
Cheat sheet

Lesson 1. Infrastructure as code and HCL basics

14 minutes. Understand why infrastructure is described in files, learn the Terraform workflow, and read the basic HCL syntax.

A teammate builds a network by clicking through a cloud console. Six months later nobody remembers which options they chose, the staging copy differs from production in ways no one can list, and rebuilding after a disaster is guesswork. Infrastructure as code (IaC) replaces memory and clicking with files that can be reviewed, repeated and tested.

What you will learn

- What infrastructure as code gives you
- The Terraform workflow: init, plan, apply, destroy
- HCL blocks, arguments and references
- How to read a plan

Why IaC

- **Repeatable**: the same files create the same result in every environment
- **Reviewable**: changes appear as a diff that a colleague approves before anything happens
- **Documented**: the files are the truth about what exists
- **Recoverable**: rebuilding is running a command, not recalling a procedure

Terraform is **declarative**. You describe the end result ("a storage bucket with these settings") and Terraform works out the steps to get there from whatever exists now.

The workflow

Command	What it does
`terraform init`	Downloads providers and modules, prepares the backend
`terraform fmt`	Formats files consistently
`terraform validate`	Checks syntax and internal consistency
`terraform plan`	Shows what would change, without changing anything
`terraform apply`	Makes the changes (asks for confirmation)
`terraform destroy`	Removes everything this configuration manages

HCL in five minutes

HCL (HashiCorp Configuration Language) is built from **blocks** that contain **arguments**:

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1"
}

resource "aws_s3_bucket" "logs" {
  bucket = "eknatha-demo-logs-12345"
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  tags = {
    Environment = "dev"
    Owner       = "platform"
  }
}

output "bucket_arn" {
  value = aws_s3_bucket.logs.arn
}
```

Read it this way:

- `terraform` pins the Terraform and provider versions, so a teammate or a pipeline gets the same behaviour.
- `provider` configures how to talk to a platform.
- `resource "TYPE" "NAME"` declares one thing to manage. The pair of type and name (`aws_s3_bucket.logs`) is its address inside Terraform.
- `aws_s3_bucket.logs.arn` is a **reference**. Terraform sees that the output depends on the bucket and orders the work correctly without being told.
- `~> 5.0` means "5.x but not 6.0".

Reading a plan

```
$ terraform init
$ terraform plan
Terraform will perform the following actions:

  # aws_s3_bucket.logs will be created
  + resource "aws_s3_bucket" "logs" {
      + bucket = "eknatha-demo-logs-12345"
      + id     = (known after apply)
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

This is abbreviated sample output. The symbols matter:

Symbol	Meaning
`+`	Create
`~`	Update in place
`-`	Destroy
`-/+`	Destroy and recreate (replacement)

Always read the summary line and look for `-` and `-/+` before applying. A replacement of a database or a load balancer is the kind of change a plan exists to catch.

> **Tip:** *Save a plan with* `terraform plan -out=tfplan` *and apply exactly that file with* `terraform apply tfplan`*. What was reviewed is then exactly what runs.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Infrastructure as code and HCL basics fits together

Part	What it does	What goes wrong
Why IaC	Repeatable: the same files create the same result in every environment.	Using the same names and state for every environment.
The workflow	.	Applying without reading the plan.
HCL in five minutes	HCL (HashiCorp Configuration Language) is built from blocks that contain arguments.	Leaving provider versions unpinned.
Reading a plan	This is abbreviated sample output.	Editing resources in the console, so reality drifts from the code.

Flow: Why IaC to The workflow / The workflow to HCL in five minutes / HCL in five minutes to Reading a plan

Incident: the plan wants to destroy the database

Incident: *A small change to a tag is proposed in review. The plan output says* `1 to add, 0 to change, 1 to destroy`*, and the destroyed resource is the production database.*

```
  # aws_db_instance.main must be replaced
-/+ resource "aws_db_instance" "main" {
      ~ identifier = "shop-prod" -> "shop-production" # forces replacement
```

Someone renamed the `identifier` while "tidying up". For that resource, a new identifier means a new database, so Terraform plans to delete the old one. The plan caught it because the reviewer read it. Revert the rename, rerun the plan, and confirm it now shows `No changes` or only the tag update. Protect the resource so this cannot happen quietly again:

```
resource "aws_db_instance" "main" {
  # ...
  lifecycle {
    prevent_destroy = true
  }
}
```

With `prevent_destroy`, any plan that would delete it fails with an error. It is a safety net, not a substitute for reading plans.

Common mistakes

- Applying without reading the plan
- Editing resources in the console, so reality drifts from the code
- Leaving provider versions unpinned
- Using the same names and state for every environment

Check yourself

What does "declarative" mean for Terraform?

Answer: You describe the desired end state and Terraform works out the steps

In a plan, what does -/+ mean?

Answer: The resource will be destroyed and recreated

What does terraform plan do?

Answer: Shows the changes that would be made without making them

What does the version constraint ~> 5.0 allow?

Answer: Versions from 5.0 up to but not including 6.0

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Providers, resources and data sources

15 minutes. Build real infrastructure with dependencies, query existing resources with data sources, and use lifecycle rules and meta-arguments safely.

A bucket is a toy. Real infrastructure has a network, subnets, security groups and servers that refer to each other. Terraform handles this by building a **dependency graph** from your references, creating things in a valid order and in parallel where it can.

What you will learn

- Providers and how they authenticate
- Resources with references and implicit dependencies
- Data sources for things that already exist
- `count`, `for_each`, `depends_on` and `lifecycle`

Providers

A provider is a plugin that knows how to manage one platform's API: AWS, Azure, Kubernetes, Cloudflare and many more. `terraform init` downloads the versions you pinned. Give providers credentials through the environment or the platform's own mechanisms (roles, workload identity), **not** through values written in your files.

```
export AWS_PROFILE=dev
terraform init
```

MARGA by EknathaLabs
marga.eknathalabs.com

A small network and server

```
resource "aws_vpc" "main" {
  cidr_block           = "10.20.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "demo" }
}

resource "aws_subnet" "app" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.20.1.0/24"
  availability_zone = "eu-west-1a"
}

resource "aws_security_group" "web" {
  name   = "web"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-2023*-x86_64"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.al2023.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.app.id
  vpc_security_group_ids = [aws_security_group.web.id]
}
```

Notice that `aws_subnet.app` uses `aws_vpc.main.id`. That reference is an **implicit dependency**: Terraform knows the VPC must exist first. A **data source** (`data "..."`) reads something that already exists or is published, here the latest machine image, without managing it.

Many of the same thing

```
variable "subnets" {
  type = map(string)
  default = {
    a = "10.20.1.0/24"
    b = "10.20.2.0/24"
  }
}

resource "aws_subnet" "multi" {
  for_each          = var.subnets
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value
  availability_zone = "eu-west-1${each.key}"
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

Meta-argument	Use
`count`	N copies indexed by number (`[0]`, `[1]`). Removing an item from the middle shifts the rest
`for_each`	One copy per key of a map or set. Each has a stable key such as `["a"]`
`depends_on`	An explicit dependency when Terraform cannot see one from references
`lifecycle`	`prevent_destroy`, `create_before_destroy`, `ignore_changes`

Prefer `for_each` over `count` for anything that is not a simple on/off switch (`count = var.enabled ? 1 : 0`). With `count`, deleting the second of three items makes Terraform rename the third, which can destroy and recreate a real resource.

Inspecting the graph and state

```
$ terraform state list
aws_vpc.main
aws_subnet.multi["a"]
aws_subnet.multi["b"]
$ terraform graph | head
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Providers, resources and data sources fits together

Part	What it does	What goes wrong
Providers	A provider is a plugin that knows how to manage one platform's API: AWS, Azure, Kubernetes, Cloudflare and many more.	Putting credentials in configuration files.
A small network and server	Notice that aws_subnet.app uses aws_vpc.main.id.	Hard-coding AMI IDs that differ per region instead of using a data source.
Many of the same thing	Prefer for_each over count for anything that is not a simple on/off switch (count = var.enabled ?	Using count for lists whose members may be removed.
Inspecting the graph and state	.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Providers to A small network and server / A small network and server to Many of the same thing / Many of the same thing to Inspecting the graph and state

Incident: the plan wants to recreate subnets after a harmless edit

> **Incident:** *You delete the first of three subnets defined with* `count`*. The plan wants to destroy and recreate the other two.*

```
  # aws_subnet.app[1] must be replaced
  # aws_subnet.app[2] must be replaced
Plan: 2 to add, 3 to destroy
```

With `count`, items are tracked by position. Removing index 0 moves the old index 1 into position 0, so Terraform sees every address as changed. Switch the resource to `for_each` over a map with stable keys, then tell Terraform that existing resources simply moved, using `moved` blocks so nothing is destroyed:

```
moved {
  from = aws_subnet.app[1]
  to   = aws_subnet.multi["b"]
}
```

Rerun the plan and confirm it shows only the intended removal. Moves are refactors of the *code*, not of the real infrastructure, and the `moved` block expresses exactly that.

Common mistakes

- Putting credentials in configuration files
- Using `count` for lists whose members may be removed
- Hard-coding AMI IDs that differ per region instead of using a data source
- Opening security groups to the whole internet on administrative ports

Check yourself

What does Terraform use to decide creation order?

Answer: A dependency graph built from references between resources

What is a data source for?

Answer: Reading information about something that already exists or is published

Why prefer for_each over count for a list of subnets?

Answer: Items have stable keys, so removing one does not shift the others

What does a moved block do?

Answer: Tells Terraform a resource address changed so it does not destroy and recreate

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Variables, locals, outputs and validation

14 minutes. Make configurations reusable with typed variables, validation, locals and outputs, and keep sensitive values out of your files.

Hard-coded values make a configuration usable exactly once. Variables turn it into something you can reuse for dev, staging and production, and validation catches bad input before a cloud API call does.

What you will learn

- Input variables with types, defaults and validation
- Ways to supply values and their order of precedence
- Locals for computed values
- Outputs, and handling sensitive values

Input variables

```
variable "environment" {
  type        = string
  description = "Deployment environment"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging or prod."
  }
}

variable "instance_count" {
  type    = number
  default = 2
}

variable "allowed_cidrs" {
  type    = list(string)
  default = ["10.0.0.0/8"]
}

variable "db_password" {
  type      = string
  sensitive = true
}
```

Always give variables a `type` and a `description`. `sensitive = true` hides the value in plan and apply output (but not from the state file, which the next lesson covers).

Supplying values

Terraform picks a value using this order, with later sources winning:

1. The variable's `default`
2. Environment variables named `TF_VAR_name`
3. `terraform.tfvars` and `*.auto.tfvars` files
4. `-var-file=` and `-var` options on the command line

```
# prod.tfvars
environment    = "prod"
instance_count = 4
```

```
$ terraform plan -var-file=prod.tfvars
$ export TF_VAR_db_password="$(cat /run/secrets/db_password)"
```

Never write real secrets into `.tfvars` files that are shared. Pass them from the environment or from a secret manager at run time.

MARGA by EknathaLabs
marga.eknathalabs.com

Locals

Locals give names to expressions so you write them once:

```
locals {
  name_prefix = "shop-${var.environment}"

  common_tags = {
    Environment = var.environment
    Owner       = "platform"
    ManagedBy   = "terraform"
  }

  is_prod = var.environment == "prod"
}

resource "aws_s3_bucket" "assets" {
  bucket = "${local.name_prefix}-assets"
  tags   = local.common_tags
}
```

Use locals for derived values and variables for inputs. Do not overuse locals to the point where readers must jump around to follow the code.

Expressions you will use constantly

Expression	Example	Result
Conditional	`local.is_prod ? "m5.large" : "t3.micro"`	Chooses a value
`for` expression	`[for s in var.names : upper(s)]`	Transforms a list
String template	`"${var.environment}-api"`	Builds a string
Functions	`merge(a, b)`, `lookup(map, key, default)`, `cidrsubnet(cidr, 8, 1)`	Built-in helpers

Use `terraform console` to try expressions without applying anything:

```
$ terraform console
> cidrsubnet("10.20.0.0/16", 8, 3)
"10.20.3.0/24"
> [for s in ["a","b"] : upper(s)]
[
  "A",
  "B",
]
```

Outputs

```
output "assets_bucket" {
  description = "Name of the assets bucket"
  value       = aws_s3_bucket.assets.bucket
}

output "db_endpoint" {
  value     = aws_db_instance.main.endpoint
  sensitive = true
}
```

Outputs print after `apply` and are how modules return values to their callers. Read one with `terraform output assets_bucket`.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Variables, locals, outputs and validation fits together

Part	What it does	What goes wrong
Input variables	Always give variables a type and a description.	Variables without types, so a string sneaks in where a number is expected.
Supplying values	Terraform picks a value using this order, with later sources winning.	Secrets in .tfvars files that are shared.
Locals	Locals give names to expressions so you write them once.	Constructing complex logic in locals that nobody can read.
Expressions you will use constantly	Use terraform console to try expressions without applying anything.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Outputs	Outputs print after apply and are how modules return values to their callers.	Forgetting that sensitive does not protect state.

Flow: Input variables to Supplying values / Supplying values to Locals / Locals to Expressions you will use constantly / Expressions you will use constantly to Outputs

Incident: a secret shows up in a pipeline log

Incident: *A reviewer notices a database password in a CI job's plan output.*

The variable had no `sensitive = true`, so Terraform printed it in the plan. First **rotate the password**, because anything printed to a log must be treated as exposed. Then fix the cause:

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

Also mark any output that exposes a derived secret as `sensitive`, and make sure the value arrives through `TF_VAR_db_password` from your CI system's secret store instead of a file. Finally, remember `sensitive` only masks display. The value still lives in the state file, so state must be stored in an encrypted backend with restricted access.

Common mistakes

- Variables without types, so a string sneaks in where a number is expected
- Secrets in `.tfvars` files that are shared
- Forgetting that `sensitive` does not protect state
- Constructing complex logic in locals that nobody can read

Check yourself

Which source of a variable value wins over an environment variable TF_VAR_x?

Answer: A -var option on the command line

What does sensitive = true on a variable do?

Answer: Hides the value in plan and apply output

What is a local value used for?

Answer: Naming an expression so it can be reused within the module

Which command lets you test an expression such as cidrsubnet without applying?

Answer: terraform console

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Expressions, loops and dynamic blocks

16 minutes. Create many resources from data with count and for_each, transform values with for expressions and functions, and generate nested blocks with dynamic.

Real infrastructure is repetitive: ten subnets, five security-group rules, three environments. Copying resource blocks is error-prone. Terraform's expression language lets you describe the pattern once and drive it from data.

What you will learn

- `count` versus `for_each`, and why the difference matters
- `for` expressions, conditionals and common functions
- `dynamic` blocks for repeated nested configuration
- Keeping configurations readable

count and for_each

`count` creates N copies addressed by index. `for_each` creates one instance per key in a map or set, addressed by that key.

```
variable "subnets" {
  type = map(object({
    cidr = string
    az   = string
  }))
  default = {
    app-a = { cidr = "10.0.1.0/24", az = "eu-west-1a" }
    app-b = { cidr = "10.0.2.0/24", az = "eu-west-1b" }
    db-a  = { cidr = "10.0.11.0/24", az = "eu-west-1a" }
  }
}

resource "aws_subnet" "this" {
  for_each          = var.subnets
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
  tags = {
    Name = "shop-${each.key}"
  }
}
```

Each subnet is tracked as `aws_subnet.this["app-a"]`. If you later remove `app-b` from the map, Terraform destroys only that subnet. With `count`, instances are tracked by position, so removing the **middle** item shifts every later index and Terraform plans to destroy and recreate resources that did not change. Prefer `for_each` whenever the items have a natural name. Use `count` mainly for a simple on/off switch: `count = var.enabled ? 1 : 0`.

for expressions and functions

A `for` expression transforms a list or map into another.

```
locals {
  names      = [for k, v in var.subnets : "${k}-${v.az}"]
  app_cidrs  = [for k, v in var.subnets : v.cidr if startswith(k, "app")]
  az_by_name = { for k, v in var.subnets : k => v.az }

  common_tags = {
    Project = "shop"
    Env     = terraform.workspace
  }
  all_tags = merge(local.common_tags, { Owner = "platform" })
}
```

MARGA by EknathaLabs marga.eknathalabs.com

```
output "app_cidrs" {
  value = local.app_cidrs
}
```

The functions you will use most: `merge` for maps, `concat` and `flatten` for lists, `lookup` with a default, `coalesce` for the first non-empty value, `cidrsubnet` for carving address ranges, `format` and `join` for strings, and `try` to fall back when an expression fails.

```
locals {
  vpc_cidr = "10.20.0.0/16"
  subnet_cidrs = [for i in range(3) : cidrsubnet(local.vpc_cidr, 8, i)]
  instance_type = lookup({ dev = "t3.small", prod = "m6i.large" }, terraform.workspace, "t3.small" \
    )
}
```

Test expressions interactively with `terraform console`:

```
$ terraform console
> cidrsubnet("10.20.0.0/16", 8, 2)
"10.20.2.0/24"
> [for s in ["a","b"] : upper(s)]
```

dynamic blocks

Some resources contain repeated nested blocks, such as ingress rules in a security group. A `dynamic` block generates one nested block per element.

```
variable "ingress_rules" {
  type = list(object({
    port = number
    cidr = string
  }))
  default = [
    { port = 443, cidr = "0.0.0.0/0" },
    { port = 22, cidr = "10.0.0.0/8" },
  ]
}

resource "aws_security_group" "web" {
  name   = "shop-web"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = "tcp"
      cidr_blocks = [ingress.value.cidr]
    }
  }
}
```

Inside `content`, the iterator is named after the block (`ingress`) and exposes `.key` and `.value`. Use dynamic blocks only when the data really varies. Overuse makes configuration hard to read, so a plain repeated block is better for two fixed rules.

Conditionals and validation

```
variable "environment" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging or prod."
  }
}

resource "aws_db_instance" "main" {
  identifier          = "shop-${var.environment}"
  instance_class      = var.environment == "prod" ? "db.m6i.large" : "db.t3.medium"
  multi_az            = var.environment == "prod"
  deletion_protection = var.environment == "prod"
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Expressions, loops and dynamic blocks fits together

Part	What it does	What goes wrong
count and for_each	count creates N copies addressed by index.	Using count over a list whose items can be removed from the middle.
for expressions and functions	A for expression transforms a list or map into another.	Not testing with terraform console and terraform plan first.
dynamic blocks	Some resources contain repeated nested blocks, such as ingress rules in a security group.	Overusing dynamic blocks so readers cannot see the resulting resource.
Conditionals and validation	.	Complex nested expressions in resource arguments instead of a named local.

Flow: count and for_each to for expressions and functions / for expressions and functions to dynamic blocks / dynamic blocks to Conditionals and validation

Incident: the plan that wants to destroy the whole fleet

Incident: *Engineers remove one server name from the list that drives* `count`*. The next* `terraform plan` *shows 14 resources to destroy and 14 to create, including production instances that did not change.*

```
$ terraform plan | grep -E "must be replaced|will be destroyed" | head
  # aws_instance.web[1] must be replaced
  # aws_instance.web[2] must be replaced
```

The list shifted, so every instance after the removed one received a new index and a new identity. Convert to `for_each` and tell Terraform the instances are the same, using `moved` blocks:

```
moved {
  from = aws_instance.web[1]
  to   = aws_instance.web["web-b"]
}
```

Run `terraform plan` again and confirm it shows no destroys before applying. Make it policy that anything keyed by names uses `for_each`.

Common mistakes

- Using `count` over a list whose items can be removed from the middle
- Using `for_each` with values not known until apply
- Overusing `dynamic` blocks so readers cannot see the resulting resource
- Complex nested expressions in resource arguments instead of a named local
- Not testing with `terraform console` and `terraform plan` first

Check yourself

Why prefer for_each over count for named items?

Answer: Instances are tracked by key, so removing one does not reindex the others

What does a dynamic block do?

Answer: Generates repeated nested blocks from a collection

What does a moved block do?

Answer: Records that a resource has a new address so Terraform does not recreate it

Which command lets you try an expression interactively?

Answer: terraform console

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. State, backends and locking

16 minutes. Understand what Terraform state is, store it remotely with locking, and use import and state commands without breaking things.

Terraform has to remember which real resources belong to which blocks in your code. That memory is the **state**. It is the most important and most dangerous file in the system: lose it and Terraform forgets what it created, corrupt it and plans become nonsense, expose it and you leak secrets.

What you will learn

- What state contains and why it matters
- Remote backends and state locking
- Importing existing resources
- Safe use of the `terraform state` commands

What state is

State maps each resource address (`aws_s3_bucket.logs`) to the real object's ID and records its attributes. During `plan`, Terraform refreshes state from the real world, compares it with your code, and computes the difference.

State also contains **every attribute**, including passwords and keys that you marked sensitive. Treat it as a secret.

> **Warning:** *Never edit the state file by hand and never share it over chat or email. Keep it out of any repository, and restrict who can read the storage that holds it.*

Remote backends and locking

A local `terraform.tfstate` file works for experiments. On a team it fails quickly: two people run `apply` at once, or one person's copy is out of date. A **remote backend** stores state centrally, and **locking** stops two runs from changing it simultaneously.

```
terraform {
  backend "s3" {
    bucket       = "acme-terraform-state"
    key          = "shop/prod/terraform.tfstate"
    region       = "eu-west-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

This stores state in an S3 bucket with encryption and uses S3's own lock file support (recent Terraform releases). Older setups use a DynamoDB table for locks, and you will still see `dynamodb_table` in existing code. For the storage itself, enable bucket **versioning** so you can recover a previous state, block public access, and allow only the pipeline and a small group to read it. Azure Storage and Terraform Cloud-style services provide the same ideas.

Use a separate `key` (or a separate bucket or account) per environment, so a mistake in dev can never touch production state.

Importing existing resources

When something already exists in the cloud, bring it under management instead of recreating it. With current Terraform, write the target in code and add an `import` block:

```
import {
  to = aws_s3_bucket.legacy
  id = "acme-legacy-reports"
}

resource "aws_s3_bucket" "legacy" {
  bucket = "acme-legacy-reports"
```

MARGA by EknathaLabs marga.eknathalabs.com

```
}
```

Run `terraform plan`, which shows the import, and `apply` to record it. Then keep running `plan` and adjust the code until it reports no changes. The older command form is `terraform import aws_s3_bucket.legacy acme-legacy-reports`.

State commands

```
$ terraform state list
$ terraform state show aws_s3_bucket.logs
$ terraform state mv aws_s3_bucket.logs aws_s3_bucket.audit_logs
$ terraform state rm aws_s3_bucket.legacy
```

Command	Effect
`state list` / `show`	Read-only inspection
`state mv`	Renames the address in state only (prefer a `moved` block, which is reviewable)
`state rm`	Makes Terraform forget a resource **without** deleting it in the cloud
`force-unlock ID`	Releases a stuck lock. Use only if you are certain no run is active

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How State, backends and locking fits together

Part	What it does	What goes wrong
What state is	State maps each resource address (aws_s3_bucket.logs) to the real object's ID and records its attributes.	One giant state for everything, so every plan is slow and a mistake can touch anything.
Remote backends and locking	A local terraform.tfstate file works for experiments.	No encryption, versioning or access control on the state bucket.
Importing existing resources	When something already exists in the cloud, bring it under management instead of recreating it.	Running force-unlock without checking for an active run.
State commands	.	Keeping state in a repository or on a laptop.

Flow: What state is to Remote backends and locking / Remote backends and locking to Importing existing resources / Importing existing resources to State commands

Incident: "Error acquiring the state lock"

Incident: *A pipeline run fails immediately with* `Error: Error acquiring the state lock`*, and the message shows a lock held by another run.*

```
Error: Error acquiring the state lock
Lock Info:
  ID:        3f6c0a1e-...
  Operation: OperationTypeApply
  Who:       ci-runner@build-17
  Created:   2026-10-24 02:14:09 UTC
```

First find out whether that run is genuinely still going. Look at your CI system for job `build-17`. If it is still applying, **wait**: unlocking during an apply is how state gets corrupted. If the job crashed or was cancelled and clearly is not running, release the lock deliberately:

```
$ terraform force-unlock 3f6c0a1e-...
```

Then run `terraform plan` to check that state and reality agree. Prevent recurrences by allowing one run per environment at a time in the pipeline, and by never killing an apply mid-way.

Predict the plan

Common mistakes

- Keeping state in a repository or on a laptop
- No encryption, versioning or access control on the state bucket
- One giant state for everything, so every plan is slow and a mistake can touch anything
- Running `force-unlock` without checking for an active run

Check yourself

What does Terraform state store?

Answer: A mapping from resource addresses to real resource IDs and their attributes

Why use a remote backend with locking?

Answer: It shares state safely and prevents concurrent runs from corrupting it

What does terraform state rm do to the real resource?

Answer: Nothing; Terraform just stops tracking it

A run holds the state lock and is still applying. What should you do?

Answer: Wait for it to finish

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Modules and project structure

15 minutes. Package reusable infrastructure as modules with clear inputs and outputs, version them, and lay out environments so changes stay isolated.

By the third environment you are copying the same hundred lines of network code and fixing the same bug in three places. A **module** is a folder of Terraform files used as a single building block. Every configuration is already a module (the "root" module). The skill is deciding what to extract and how to keep it clean.

What you will learn

- How to write and call a module
- Good interfaces: inputs, outputs and sensible defaults
- Versioning and sourcing modules
- A directory layout for several environments

A module

```
modules/
  static-site/
    main.tf
    variables.tf
    outputs.tf
```

```
# modules/static-site/variables.tf
variable "name" {
  type        = string
  description = "Prefix for all resource names"
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

```
# modules/static-site/main.tf
resource "aws_s3_bucket" "site" {
  bucket = "${var.name}-site"
  tags   = var.tags
}

resource "aws_s3_bucket_public_access_block" "site" {
  bucket                  = aws_s3_bucket.site.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

```
# modules/static-site/outputs.tf
output "bucket_name" {
  value = aws_s3_bucket.site.bucket
}
```

Calling it:

```
module "marketing" {
  source = "../../modules/static-site"

  name = "marketing-prod"
  tags = { Owner = "web" }
}

output "marketing_bucket" {
  value = module.marketing.bucket_name
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

After adding a module call, run `terraform init` again so Terraform can load it. A module can only be read through its **outputs**; callers cannot reach resources inside it directly.

Designing a good module

- **Small, clear purpose.** "A VPC with subnets" is a module. "Everything for our company" is not.
- **Few required inputs, safe defaults.** The secure choice should be the default.
- **Typed and described variables.** The interface is the documentation.
- **Useful outputs.** Return IDs and names that callers need to connect other things.
- **No provider configuration inside.** Let the caller configure providers, so the module works in any account or region.
- **Do not wrap a single resource without adding value.** A module that just renames one resource's arguments adds complexity only.

Versions and sources

Source	Example	Use
Local path	`source = "../../modules/vpc"`	Same repository, changes together
Registry	`source = "terraform-aws-modules/vpc/aws"` with `version = "~> 5.0"`	Well-maintained community modules
Internal registry or archive	A versioned release your team publishes	Shared across many teams

Always **pin versions** of modules you do not control, and read what a community module does before depending on it. An unpinned module can change under you on the next `init -upgrade`.

Layout for environments

```
infra/
  modules/
    network/
    app/
  envs/
    dev/
      main.tf
      backend.tf
      dev.tfvars
    prod/
      main.tf
      backend.tf
      prod.tfvars
```

Each environment is its own root module with its **own state**. They call the same modules with different inputs. A change to dev cannot touch production, and you promote a module change by updating the version or path in each environment in turn. Avoid using one set of files with `terraform workspace` to separate production from development. Workspaces share code and backend configuration, which makes it easy to run a command against the wrong one.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Modules and project structure fits together

Part	What it does	What goes wrong
A module	Calling it.	Renaming resources inside a module without a moved block.
Designing a good module	Small, clear purpose. "A VPC with subnets" is a module.	Giant modules that do everything.
Versions and sources	Always pin versions of modules you do not control, and read what a community module does before depending on it.	Unpinned remote modules.
Layout for environments	Each environment is its own root module with its own state.	Environments that share one state.

Flow: A module to Designing a good module / Designing a good module to Versions and sources / Versions and sources to Layout for environments

Incident: a module change breaks every environment at once

> **Incident:** *A fix to the shared* `network` *module is merged. The next scheduled run in production proposes to replace the subnets.*

The environments referenced the module by local path, so they all picked up the change immediately, and the new code renamed an internal resource. The plan flagged it before any damage:

```
  # module.network.aws_subnet.private[0] must be replaced
Plan: 3 to add, 0 to change, 3 to destroy.
```

Do not apply. Revert or correct the module change, and for a rename add a `moved` block inside the module so existing resources keep their identity. To avoid the class of problem, publish modules as **versioned releases** and let each environment upgrade deliberately: dev first, then staging, then production, reading the plan at each step.

Common mistakes

- Giant modules that do everything
- Unpinned remote modules
- Environments that share one state
- Provider blocks inside reusable modules
- Renaming resources inside a module without a `moved` block

Check yourself

How does a caller read a value from inside a module?

Answer: Through the module's outputs, such as module.marketing.bucket_name

Why should reusable modules not contain provider configuration?

Answer: The caller should decide the account and region so the module is reusable

What must you do after adding a new module call?

Answer: Run terraform init so the module is loaded

Why give each environment its own state?

Answer: So a change or mistake in one environment cannot affect another

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. Secrets and sensitive data in Terraform

14 minutes. Keep passwords and keys out of code and logs, understand that state holds secrets, read secrets from a manager at runtime, and secure the state backend.

Terraform needs to handle database passwords, API keys and certificates, yet its design stores a great deal in plain text. Knowing exactly where secrets can leak is the first step to preventing it.

What you will learn

- Why secrets appear in state and plan output
- Marking values `sensitive` and what that does and does not do
- Generating and fetching secrets from a secret manager
- Protecting state, and ephemeral approaches

Where secrets leak

1. **Source code**: a password written in a `.tf` file or in `terraform.tfvars` committed to version control
2. **Plan and apply output**: values shown in the console or CI log
3. **State**: resource attributes, including passwords you passed in, are stored in the state file in plain text
4. **Environment and shell history**: variables exported or typed on the command line

The `sensitive = true` setting hides a value from CLI output, but it does **not encrypt it in state**.

```
variable "db_password" {
  type      = string
  sensitive = true
}

output "connection_string" {
  value     = "postgres://app:${var.db_password}@${aws_db_instance.main.address}/shop"
  sensitive = true
}
```

Anyone who can read the state file can read the secret. That is why state must be treated as a secret in itself.

Do not choose passwords in code

Let the platform generate a password and keep it in a secret manager.

```
resource "random_password" "db" {
  length  = 32
  special = false
}

resource "aws_secretsmanager_secret" "db" {
  name = "shop/prod/db"
}

resource "aws_secretsmanager_secret_version" "db" {
  secret_id     = aws_secretsmanager_secret.db.id
  secret_string = random_password.db.result
}

resource "aws_db_instance" "main" {
  identifier          = "shop-prod"
  engine              = "postgres"
  instance_class      = "db.t3.medium"
  allocated_storage   = 50
  username            = "app"
  password            = random_password.db.result
  skip_final_snapshot = false
}
```

MARGA by EknathaLabs marga.eknathalabs.com

The password still passes through state, but it never appears in code, tickets or chat. Some resources support **managed passwords** where the service creates and rotates the secret itself (for example `manage_master_user_password = true` on RDS), which keeps the value out of Terraform entirely. Prefer that when available.

Reading existing secrets

```
data "aws_secretsmanager_secret_version" "api_key" {
  secret_id = "shop/prod/payments-api-key"
}

resource "aws_lambda_function" "worker" {
  function_name = "shop-worker"
  role          = aws_iam_role.worker.arn
  runtime       = "python3.12"
  handler       = "main.handler"
  filename      = "worker.zip"

  environment {
    variables = {
      PAYMENTS_SECRET_ARN = "shop/prod/payments-api-key"
    }
  }
}
```

Notice the better pattern in the example: pass the **name or ARN** of the secret to the application and let the application fetch it at runtime with its IAM role. That keeps the value out of the Terraform state and out of the Lambda configuration. Reading a secret with a `data` source writes the value into state.

Securing the state backend

- Store state in a remote backend with encryption at rest (for example an S3 bucket with SSE-KMS, or Azure Storage with encryption)
- Enable versioning so you can recover from corruption
- Restrict access by IAM: only the pipeline role and a few administrators can read state
- Enable locking, block public access and turn on access logging
- Never commit `*.tfstate`, `*.tfvars` with secrets or `.terraform/` to version control

```
$ terraform state pull | jq '.resources[] | select(.type=="aws_db_instance") | .instances[0].attri \
    butes.password'
$ aws s3api get-bucket-encryption --bucket acme-tfstate
$ aws s3api get-public-access-block --bucket acme-tfstate
```

The second command demonstrates the risk: reading the state exposes the database password.

Supplying inputs in a pipeline

```
$ export TF_VAR_db_password="$(aws secretsmanager get-secret-value --secret-id shop/prod/db --quer \
    y SecretString --output text)"
$ terraform plan -out=tfplan
$ terraform apply tfplan
```

Plan files can also contain secrets, so treat them as sensitive and delete them after use. CI systems should inject variables from a protected store and mask them in logs.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Secrets and sensitive data in Terraform fits together

Part	What it does	What goes wrong
Where secrets leak	Source code: a password written in a .tf file or in terraform.tfvars committed to version control.	Believing sensitive = true encrypts the state.
Do not choose passwords in code	Let the platform generate a password and keep it in a secret manager.	Committing terraform.tfvars or state files.
Reading existing secrets	Notice the better pattern in the example: pass the name or ARN of the secret to the application and let the application fetch it at runtime with its IAM role.	Reading secrets with data sources and passing them into resource arguments unnecessarily.
Securing the state backend	Store state in a remote backend with encryption at rest (for example an S3 bucket with SSE-KMS, or Azure Storage with encryption).	Leaving the state bucket readable by many people.
Supplying inputs in a pipeline	Plan files can also contain secrets, so treat them as sensitive and delete them after use.	Keeping old plan files that contain secrets.

Flow: Where secrets leak to Do not choose passwords in code / Do not choose passwords in code to Reading existing secrets / Reading existing secrets to Securing the state backend / Securing the state backend to Supplying inputs in a pipeline

Incident: the password in the pull request

> **Incident:** *A developer adds* `password = "S3cret!2024"` *to a Terraform file and opens a pull request. The branch is pushed, the CI log prints the plan, and the password appears in the review tool, the build log and the plan artifact.*

Treat the password as compromised. Rotate it first, then clean up:

```
$ aws rds modify-db-instance --db-instance-identifier shop-prod --manage-master-user-password --ap \
    ply-immediately
$ aws secretsmanager get-secret-value --secret-id 'rds!db-abc123' --query SecretString --output te \
    xt | head -c 0
```

After rotation, remove the literal from code, mark the variable `sensitive`, delete or redact the log and artifact, and add a pre-commit secret scanner plus a pipeline check that fails when credentials patterns appear. Prefer managed passwords so there is nothing to commit.

Common mistakes

- Believing `sensitive = true` encrypts the state
- Committing `terraform.tfvars` or state files
- Reading secrets with data sources and passing them into resource arguments unnecessarily
- Leaving the state bucket readable by many people
- Keeping old plan files that contain secrets

Check yourself

Does marking a variable sensitive encrypt it in the state file?

Answer: No, it only hides the value from CLI output

What is the best way for an application to get a database password?

Answer: Fetch it from a secret manager at runtime using its own identity

Which backend practice protects state?

Answer: Encryption at rest, versioning, locking and restricted access

What should you do first when a secret is committed?

Answer: Rotate or revoke the secret

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. End to end: a web stack on AWS and on Azure

18 minutes. Build the same small web platform with Terraform on AWS and on Azure, network, compute, load balancer and database, and compare the equivalent resources.

Concepts become clear when you build something. This lesson assembles a small, realistic stack on both clouds: a private network, a load balancer, application servers and a managed database. The resources differ in name, but the design is the same.

What you will learn

- A reference layout: public edge, private application tier, private data tier
- The AWS resources and their Azure equivalents
- How to structure the project, remote state and variables
- A safe plan, apply and destroy routine

The design

Layer	AWS	Azure
Network	VPC, subnets, route tables	Virtual network, subnets, route tables
Internet access	Internet gateway, NAT gateway	Public IP, NAT gateway
Traffic filtering	Security groups	Network security groups
Edge	Application Load Balancer	Application Gateway or Load Balancer
Compute	EC2 or Auto Scaling group	Virtual machine scale set
Database	RDS PostgreSQL	Azure Database for PostgreSQL flexible server
Identity	IAM role on the instance	Managed identity

Only the load balancer is public. Application servers sit in private subnets and the database in its own subnets, reachable only from the application tier.

Project layout

```
infra/
  aws/
    main.tf
    variables.tf
    outputs.tf
    backend.tf
  azure/
    main.tf
    variables.tf
    outputs.tf
    backend.tf
  modules/
    network/
```

Keep each cloud in its own root module with its own state. Do not try to make one configuration create both: the providers, authentication and lifecycle differ.

MARGA by EknathaLabs marga.eknathalabs.com

AWS network and compute

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket         = "acme-tfstate"
    key            = "shop/aws/terraform.tfstate"
    region         = "eu-west-1"
    dynamodb_table = "tf-locks"
    encrypt        = true
  }
}

provider "aws" {
  region = "eu-west-1"
}

resource "aws_vpc" "main" {
  cidr_block           = "10.10.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "shop" }
}

resource "aws_subnet" "private" {
  count             = 2
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index + 10)
  availability_zone = data.aws_availability_zones.available.names[count.index]
}

data "aws_availability_zones" "available" {
  state = "available"
}

resource "aws_security_group" "app" {
  name   = "shop-app"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port       = 8080
    to_port         = 8080
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "alb" {
  name   = "shop-alb"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

The application security group allows traffic **only from the load balancer's security group**, not from an address range. That is the AWS idiom for tier-to-tier rules.

MARGA by EknathaLabs
marga.eknathalabs.com

Azure network and compute

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.100"
    }
  }
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "acmetfstate"
    container_name       = "tfstate"
    key                  = "shop/azure.tfstate"
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "shop" {
  name     = "rg-shop-prod"
  location = "westeurope"
}

resource "azurerm_virtual_network" "main" {
  name                = "vnet-shop"
  address_space       = ["10.20.0.0/16"]
  location            = azurerm_resource_group.shop.location
  resource_group_name = azurerm_resource_group.shop.name
}

resource "azurerm_subnet" "app" {
  name                 = "snet-app"
  resource_group_name  = azurerm_resource_group.shop.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.20.10.0/24"]
}

resource "azurerm_network_security_group" "app" {
  name                = "nsg-app"
  location            = azurerm_resource_group.shop.location
  resource_group_name = azurerm_resource_group.shop.name

  security_rule {
    name                       = "allow-lb-8080"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "8080"
    source_address_prefix      = "AzureLoadBalancer"
    destination_address_prefix = "*"
  }
}
```

Azure resources live in a **resource group**, which acts as a lifecycle and access boundary. Network security groups use numbered priorities where the lowest number wins.

Running it safely

```
$ cd infra/aws
$ terraform init
$ terraform fmt -check && terraform validate
$ terraform plan -out=tfplan
$ terraform show tfplan | head -40
$ terraform apply tfplan
$ terraform output
$ terraform destroy -target=aws_instance.test
```

MARGA by EknathaLabs
marga.eknathalabs.com

Read the plan every time. Look for `-/+` (replace) and `destroy` lines on stateful resources such as databases. Use `lifecycle { prevent_destroy = true }` on the database and enable deletion protection. Tag everything with owner and environment so cost reports make sense.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How End to end: a web stack on AWS and on Azure fits together

Part	What it does	What goes wrong
The design	Only the load balancer is public. Application servers sit in private subnets and the database in its own subnets, reachable only from the application tier.	Wide-open security groups and storage accounts for convenience.
Project layout	Keep each cloud in its own root module with its own state.	One giant state for everything.
AWS network and compute	The application security group allows traffic only from the load balancer's security group, not from an address range.	Not pinning provider versions.
Azure network and compute	Azure resources live in a resource group, which acts as a lifecycle and access boundary.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Running it safely	Read the plan every time. Look for -/+ (replace) and destroy lines on stateful resources such as databases.	Forgetting that destroy is permanent for databases without snapshots.

Flow: The design to Project layout / Project layout to AWS network and compute / AWS network and compute to Azure network and compute / Azure network and compute to Running it safely

Incident: the apply that replaced the database

> **Incident:** *A change renames an RDS argument. The plan shows* `aws_db_instance.main must be replaced`*, but the engineer scrolls past it in a 400-line plan, approves it, and the database is destroyed and recreated empty.*

Prevention starts in code and process:

```
resource "aws_db_instance" "main" {
  identifier              = "shop-prod"
  engine                  = "postgres"
  instance_class          = "db.t3.medium"
  allocated_storage       = 50
  username                = "app"
  manage_master_user_password = true
  deletion_protection     = true
  backup_retention_period = 14
  skip_final_snapshot     = false
  final_snapshot_identifier = "shop-prod-final"

  lifecycle {
    prevent_destroy = true
  }
}
```

With `prevent_destroy` the plan itself errors instead of destroying. Recover from the automated backups or the final snapshot, add a CI step that fails when a plan contains a replacement of a protected resource, and require a second reviewer for any plan that destroys.

Common mistakes

- One giant state for everything
- Wide-open security groups and storage accounts for convenience
- Not pinning provider versions

MARGA by EknathaLabs marga.eknathalabs.com

- Applying a plan different from the one reviewed
- Forgetting that destroy is permanent for databases without snapshots

Check yourself

What is an idiomatic way to allow only the load balancer to reach the app tier on AWS?

Answer: Reference the load balancer's security group as the source

What is an Azure resource group?

Answer: A container that holds related resources and acts as a lifecycle and access boundary

What does prevent_destroy do?

Answer: Makes any plan that would destroy the resource fail with an error

Why keep AWS and Azure in separate root modules?

Answer: They have different providers, authentication and lifecycles, and separate state limits the blast radius

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Team workflows, drift and testing

15 minutes. Run Terraform safely in a pipeline with reviewed plans, detect and resolve drift, and add automated checks before apply.

Running `terraform apply` from a laptop works until the second engineer joins. Then you need a shared workflow: changes reviewed before they run, one place where applies happen, and a way to notice when reality has wandered away from the code.

What you will learn

- A pipeline pattern for plan and apply
- Detecting and handling drift
- Static checks and tests before apply
- Safe handling of destructive changes

The pipeline pattern

1. A change is proposed and a pipeline runs `fmt -check`, `validate` and security checks.
2. The pipeline runs `plan` and posts the result for review.
3. A human approves the plan and the code together.
4. After approval, the pipeline runs `apply` with the **saved** plan, from a single controlled environment.

```
terraform fmt -check -recursive
terraform init -input=false
terraform validate
terraform plan -input=false -out=tfplan
# review step happens here
terraform apply -input=false tfplan
```

The pipeline should authenticate with short-lived credentials (a role assumed through the CI system's identity federation) and not long-lived keys stored as variables. Give plan jobs read-only permission and apply jobs the write permission, so a plan on an untrusted change cannot modify anything.

Drift

Drift means the real infrastructure no longer matches the code. It happens when someone changes something in the console, when another tool modifies a resource, or when a cloud service changes a default.

```
$ terraform plan -detailed-exitcode
$ echo $?
2
```

With `-detailed-exitcode`, the exit status is `0` for no changes, `1` for an error, and `2` when changes are present. Run this on a schedule for each environment and alert on `2` where nothing was supposed to change. When you find drift, decide which side is right:

Situation	Action
The manual change was a mistake	`apply` to put the real resource back to the code
The manual change was needed	Update the code to match, then confirm `plan` is clean
The attribute is managed elsewhere (an autoscaler changes the count)	Add `lifecycle { ignore_changes = [...] }` for that field only

`terraform apply -refresh-only` updates state to match reality without changing infrastructure, useful when you want to record a change that is already correct.

MARGA by EknathaLabs
marga.eknathalabs.com

Checks before apply

Check	Catches
`terraform fmt -check`	Style inconsistencies
`terraform validate`	Syntax and reference errors
`tflint`	Provider-specific mistakes such as invalid instance types
`trivy config .` or `checkov`	Insecure settings such as open security groups and unencrypted storage
`terraform test`	Behaviour of modules against assertions you write
Policy checks (for example OPA)	Organization rules, such as required tags

A minimal native test file:

```
# tests/bucket.tftest.hcl
run "bucket_is_private" {
  command = plan

  variables {
    name = "test-site"
  }

  assert {
    condition     = aws_s3_bucket_public_access_block.site.block_public_acls == true
    error_message = "Public ACLs must be blocked."
  }
}
```

Destructive changes

Treat these as separate, deliberate changes: replacing databases, deleting storage, and changing network ranges. Use `prevent_destroy`, take backups or snapshots first, apply in a quiet period, and confirm the plan lists exactly the resources you expect. For zero-downtime replacement of stateless resources, `create_before_destroy` builds the new one first.

Incident: scheduled plan shows unexpected changes

Incident: *Your nightly drift check exits with 2 and the plan wants to remove a security group rule that nobody in the team touched.*

```
# aws_security_group.web will be updated in-place
~ resource "aws_security_group" "web" {
    ~ ingress = [
        - {
            - cidr_blocks = ["203.0.113.10/32"]
            - description = "temp debug access"
            - from_port   = 22
```

Someone added an SSH rule in the console during an investigation. The drift check found it. Do not just apply and move on. Ask who added it and why, since an incident may still be in progress. If the access is no longer needed, apply to remove it, which restores the intended state and closes an unreviewed hole. If it is needed, add it to the code through a reviewed change. Then reduce future console edits: restrict write permission in the console to a break-glass role, and make that role's use alert the team.

Diagram: A safe infrastructure change

Part	What it does	What goes wrong
Pull request	Code change and a plan comment.	Without a plan, reviewers guess.
Plan	Shows adds, changes and destroys.	A destroy of a database must stop everything.
Review	A person checks the plan, not only the code.	Rubber-stamping misses surprises.
Apply	Applies the saved plan from the pipeline.	Applying from a laptop skips audit and locking.
Drift check	A scheduled plan finds manual changes.	Unnoticed drift makes the next apply risky.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Pull request to Plan / Plan to Review / Review to Apply / Apply to Drift check / Drift check to Pull request

Common mistakes

- Applying a fresh plan that differs from the one reviewed
- Long-lived cloud keys stored in CI
- Never checking for drift
- Disabling a failing security check instead of fixing the finding

Check yourself

What does terraform plan -detailed-exitcode return when changes are pending?

Answer: 2

Why apply a saved plan file in a pipeline?

Answer: What was reviewed is exactly what is applied

What is drift?

Answer: Real infrastructure differing from what the code and state describe

An autoscaler legitimately changes a resource's desired count. How do you stop Terraform fighting it?

Answer: lifecycle { ignore_changes = [desired_count] } on that attribute

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Testing Terraform code

14 minutes. Catch mistakes early with formatting, validation, linting, policy checks, plan assertions and real test deployments using terraform test and Terratest, in a pipeline that stays affordable.

Infrastructure code changes can delete databases, open networks to the world or cost a lot of money. Yet many teams test it only by applying it and watching. A layered approach finds most mistakes in seconds, before anything is created, and keeps expensive tests for the cases that need them.

What you will learn

- The testing layers from cheap to expensive
- `fmt`, `validate`, linters and security scanners
- Policy checks on the plan
- `terraform test` with mocks and real runs
- Keeping test environments cheap and clean

Layers

Layer	Tool	Finds	Cost
Format	`terraform fmt -check`	Style differences	Seconds
Validate	`terraform validate`	Syntax, wrong references, wrong types	Seconds
Lint	`tflint`	Invalid instance types, deprecated syntax, provider rules	Seconds
Security scan	Trivy, Checkov or similar	Public buckets, open ports, missing encryption	Seconds
Plan checks	Policy as code on plan JSON	Rules such as required tags, no destroys of tagged resources	Seconds to a minute
Unit tests	`terraform test` with mocks	Module logic and outputs	Seconds
Integration tests	`terraform test` or Terratest with real resources	Whether it really works	Minutes, small cost

The cheap layers in a pipeline

```
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
tflint --recursive
trivy config .
terraform plan -out=tfplan && terraform show -json tfplan > plan.json
conftest test plan.json --policy policy/
```

Run these on every change. They need no cloud credentials for most steps (use `-backend=false` and, for the plan, a read-only role).

terraform test

Terraform has a built-in test framework with files that end in `.tftest.hcl`:

```
# tests/bucket.tftest.hcl
variables {
  name = "test-logs"
}

run "creates_private_encrypted_bucket" {
  command = plan
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  assert {
    condition     = aws_s3_bucket_public_access_block.this.block_public_acls == true
    error_message = "public ACLs must be blocked"
  }

  assert {
    condition     = length(aws_s3_bucket.this.tags) >= 2
    error_message = "bucket must carry owner and environment tags"
  }
}
```

With `command = plan`, nothing is created. With `command = apply` (the default), Terraform creates real resources in the test and destroys them at the end. Mocked providers let you test module logic without any cloud calls.

Integration tests

Use a separate account or project with budget alerts, unique names (random suffix) so tests can run in parallel, and a clean-up job that removes leftovers older than a few hours. Go-based Terratest can also check behaviour, such as making an HTTP request to the new load balancer. Test modules that are used widely; do not apply a full production stack on every commit.

What to assert

- Security: encryption on, public access off, least-privilege policies.
- Contracts: a module's outputs exist and have the expected shape.
- Safety: a plan for a change to production does not destroy tagged data stores.
- Defaults: a module works with only the required inputs.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Testing Terraform code fits together

Part	What it does	What goes wrong
Layers	.	Tests that leave resources behind.
The cheap layers in a pipeline	Run these on every change. They need no cloud credentials for most steps (use -backend=false and, for the plan, a read-only role).	Only testing by applying to production.
terraform test	Terraform has a built-in test framework with files that end in .tftest.hcl.	Integration tests on every commit in an expensive account.
Integration tests	Use a separate account or project with budget alerts, unique names (random suffix) so tests can run in parallel, and a clean-up job that removes leftovers older than a...	Policy checks that can be skipped by developers.
What to assert	Security: encryption on, public access off, least-privilege policies.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Layers to The cheap layers in a pipeline / The cheap layers in a pipeline to terraform test / terraform test to Integration tests / Integration tests to What to assert

Incident: the module that opened the database

Incident: *A module update makes a database reachable from the internet in a staging account. It is found two weeks later by an external scan.*

```
$ conftest test plan.json --policy policy/
FAIL - plan.json - aws_security_group.db: ingress from 0.0.0.0/0 is not allowed
```

MARGA by EknathaLabs marga.eknathalabs.com

The new default for a variable had changed the allowed range, and no test examined the plan. The team added a plan policy that rejects public ingress to data stores, a `terraform test` for the module defaults, and made the checks required for merging.

Common mistakes

- Only testing by applying to production
- Tests that leave resources behind
- Integration tests on every commit in an expensive account
- Policy checks that can be skipped by developers

Check yourself

Which command checks syntax and references without calling the cloud?

Answer: terraform validate

What does command = plan in a terraform test run block do?

Answer: Checks assertions against a plan without creating resources

Why use unique random names in integration tests?

Answer: So parallel runs do not collide and leftovers are easy to find

What is the benefit of policy checks on the plan JSON?

Answer: Rules such as no public ingress are enforced before anything is created

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Environments, repository layout and team workflows

14 minutes. Choose between workspaces and directories for environments, split state to limit blast radius, and set up reviewed, automated plan and apply flows for a team.

A Terraform setup that works for one person fails when a team shares it. Two people apply at once, a change to a test environment accidentally touches production, or state grows so big that every plan takes ten minutes. The layout you choose and the workflow around it decide how safe and fast the team can be.

What you will learn

- Workspaces compared with directories per environment
- How to split state and why
- Sharing outputs between stacks
- Pipeline flows: plan on review, apply on merge
- Roles, credentials and approvals

One code base, several environments

Workspaces give one configuration multiple state files in the same backend. They are quick for identical copies (for example, temporary preview environments), but the code cannot differ much, and it is easy to forget which workspace is selected.

Directories per environment (`envs/dev`, `envs/staging`, `envs/prod`) each call shared modules with their own variables and backend. They are explicit, can use different provider settings and credentials, and make a diff show exactly which environment changes. Most teams prefer this for long-lived environments.

```
infra/
  modules/
    network/
    app/
  envs/
    dev/      main.tf  backend.tf  terraform.tfvars
    staging/  main.tf  backend.tf  terraform.tfvars
    prod/     main.tf  backend.tf  terraform.tfvars
```

Use versioned modules (a tag or registry version) so that production uses a known version while you try a newer one in development.

Split state to limit blast radius

One state for everything is slow and risky: any plan reads every resource, and a bad apply can touch everything. Split by lifecycle and ownership: networking, shared data stores, and each application in its own state. Rule of thumb: things that change together and are owned by one team belong together; things that change at different speeds belong apart.

Share values between stacks with outputs and `terraform_remote_state`, or better, by reading data sources (look up the VPC by tag) so stacks stay loosely coupled.

Team workflow

1. A change is made on a branch and a pull request opens.
2. The pipeline runs format, validate, lint, security checks and `terraform plan`, and posts the plan as a comment.
3. A reviewer reads the plan, not just the code.
4. After approval and merge, the pipeline applies the saved plan for that environment, in order: dev, staging, then production with an approval.
5. Only the pipeline's role may apply to production; people have read-only access.

MARGA by EknathaLabs marga.eknathalabs.com

Use remote state with locking and encryption, one pipeline job at a time per state, and `-lock-timeout` to wait instead of failing. Apply the exact plan file that was reviewed (`terraform apply tfplan`) so what was approved is what runs.

Credentials

Prefer short-lived credentials from the pipeline's identity (OpenID Connect to the cloud provider) over stored keys. Use a separate role per environment with the least privilege needed. Keep secrets out of variables files and state outputs where possible, and mark sensitive values.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Environments, repository layout and team workflows fits together

Part	What it does	What goes wrong
One code base, several environments	Workspaces give one configuration multiple state files in the same backend.	Using workspaces for very different environments.
Split state to limit blast radius	One state for everything is slow and risky: any plan reads every resource, and a bad apply can touch everything.	Applying a different plan than the reviewed one.
Team workflow	A change is made on a branch and a pull request opens.	One giant state.
Credentials	Prefer short-lived credentials from the pipeline's identity (OpenID Connect to the cloud provider) over stored keys.	Long-lived keys in the pipeline.

Flow: One code base, several environments to Split state to limit blast radius / Split state to limit blast radius to Team workflow / Team workflow to Credentials

Incident: the shared state that took four hours

Incident: *An urgent security group change needs four hours because plans on the single shared state take 25 minutes and conflict with other people's applies.*

```
$ time terraform plan
...
real    24m12s
$ terraform state list | wc -l
3180
```

All environments' resources lived in one state. The team split networking, data stores and each service into separate states with their own pipelines. Plans fell to about a minute, applies no longer collided, and a change to one service could not alter the network.

Common mistakes

- Using workspaces for very different environments
- One giant state
- Applying a different plan than the reviewed one
- Long-lived keys in the pipeline

Check yourself

When are directories per environment usually better than workspaces?

Answer: When environments differ in settings, credentials or provider configuration

Why split a large state?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: To shorten plans and limit the blast radius of a mistake

What should the pipeline apply to production?

Answer: The saved plan file that reviewers approved

Which credential approach is preferred for pipelines?

Answer: Short-lived credentials from the pipeline identity

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Importing resources and refactoring safely

13 minutes. Bring existing infrastructure under Terraform, rename and reorganize resources without destroying them, and use plan to prove it.

Half of your infrastructure was created by hand in the console, and the other half sits in Terraform code that you now want to reorganize into modules. Both jobs have the same danger: Terraform decides what to destroy by comparing addresses in state with addresses in code. If those do not line up, the plan wants to delete real things.

What you will learn

- Import existing resources with the import block
- Rename and move resources with moved blocks
- Remove a resource from management without destroying it
- Always read the plan: zero destroys is the goal

Import existing resources

An `import` block tells Terraform that a resource in the cloud already exists and should be tracked at a given address. It is code, so it can be reviewed, unlike an ad hoc command.

```
import {
  to = aws_s3_bucket.assets
  id = "acme-assets-prod"
}

resource "aws_s3_bucket" "assets" {
  bucket = "acme-assets-prod"
}
```

Run `terraform plan` and the plan shows the import. Then add attributes to the resource until the plan shows no changes for it, because Terraform compares your code to the real settings. `terraform plan -generate-config-out=generated.tf` can write a first draft of the resource for you to clean up. Imports update state only; they never change the real resource until you apply a change.

Rename and move with moved blocks

If you rename `aws_instance.web` to `aws_instance.app`, Terraform sees one resource to destroy and a new one to create. A `moved` block says it is the same object.

```
moved {
  from = aws_instance.web
  to   = aws_instance.app
}
```

The same works when you move a resource into a module: `from = aws_instance.app` and `to = module.compute.aws_instance.app`. The plan then reports "has moved" and no destroy. Keep moved blocks in the code until every environment has applied them, because each state needs to see the move once. Compared with `terraform state mv`, a moved block is reviewed, repeatable, and works the same for every environment.

Stop managing without destroying

To hand a resource to another team or tool, remove it from state without touching the real object. The `removed` block does this in code:

```
removed {
  from = aws_s3_bucket.legacy
  lifecycle {
    destroy = false
  }
}
```

MARGA by EknathaLabs marga.eknathalabs.com

Without `destroy = false`, deleting the resource block would plan a destroy. Protect critical resources with `prevent_destroy = true` in `lifecycle`, so that an accidental removal fails at plan time.

Prove the refactor

The proof is the plan. Before and after a refactor, run `terraform plan` and read the summary line. A pure refactor should say `0 to add, 0 to change, 0 to destroy`, with only "moved" or "import" notes. Do it first on a non-production environment with the same code, then in production through the normal pipeline, with the plan attached to the review. If anything is to be destroyed that you did not intend, stop and fix the addressing before applying.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Importing resources and refactoring safely fits together

Part	What it does	What goes wrong
Import existing resources	An import block tells Terraform that a resource in the cloud already exists and should be tracked at a given address.	Importing a resource and applying before the plan shows no changes.
Rename and move with moved blocks	If you rename aws_instance.web to aws_instance.app, Terraform sees one resource to destroy and a new one to create.	Removing the moved block before every environment has applied it.
Stop managing without destroying	To hand a resource to another team or tool, remove it from state without touching the real object.	Renaming resources without a moved block.
Prove the refactor	The proof is the plan. Before and after a refactor, run terraform plan and read the summary line.	Forgetting prevent_destroy on databases and networks.

Flow: Import existing resources to Rename and move with moved blocks / Rename and move with moved blocks to Stop managing without destroying / Stop managing without destroying to Prove the refactor

Incident: The plan that wanted to delete production

> **Incident:** *A cleanup moved all network resources into a module, and the pipeline plan showed 38 resources to destroy, including the production VPC.*

```
$ terraform plan | tail -4
Plan: 38 to add, 0 to change, 38 to destroy.
  # aws_vpc.main will be destroyed
  # module.network.aws_vpc.main will be created
```

The engineer had tested only in a scratch account that had no state. Reviewers noticed the destroy lines, the change was stopped, and moved blocks were added for each resource. The plan then read "38 moved" with no destroys. The team added a pipeline rule that fails when a plan destroys resources tagged `tier=production`, unless a reviewer adds an explicit approval label.

Common mistakes

- Renaming resources without a moved block
- Removing the moved block before every environment has applied it
- Importing a resource and applying before the plan shows no changes
- Forgetting prevent_destroy on databases and networks

Check yourself

What does an import block do?

Answer: Adds an existing real resource to state at a given address

What does a moved block tell Terraform?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: The resource at the old address is the same object as at the new address

How do you stop managing a resource without destroying it?

Answer: Use a removed block with destroy = false

What is the best proof that a refactor is safe?

Answer: A plan with no destroys, only moved or import notes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Large configurations: structure, speed and blast radius

12 minutes. Split big Terraform code into stacks that plan quickly and fail small, with clear boundaries and dependencies between them.

One Terraform project grew to 1,800 resources. A plan takes eleven minutes, two engineers block each other on the state lock, and a small change to a tag shows a plan that touches the network. Size is the cause of all three. The cure is splitting by risk and by owner, not by file name.

What you will learn

- Why one giant state is slow and risky
- How to split into stacks by lifecycle and ownership
- Sharing values between stacks safely
- Making plans faster: targeted refresh, parallelism and provider settings

The cost of one big state

Terraform reads every resource in the state during a plan to see if it changed, and each read is an API call. More resources means a longer plan and more rate limit pressure. The state lock allows one writer at a time, so a big shared state serializes the whole team. And the **blast radius** grows: a mistake in a shared file can affect networking, databases and applications together.

Split by lifecycle and owner

Put things that change at different speeds, or belong to different teams, in separate stacks, each with its own state:

```
stacks/
  foundation/    accounts, DNS zones, identity      (changes rarely)
  network/       VPCs, subnets, routing             (platform team)
  data/          databases, queues, buckets         (data team)
  apps/shop/     services, load balancers, autoscaling (shop team)
```

A good boundary is one where the lower stack changes much less often than the upper one, and where a different group would approve changes. Avoid splitting into dozens of tiny stacks with circular dependencies, because every connection between them is something you must keep consistent by hand.

Share values between stacks

A stack exposes **outputs**, and others read them. The two common ways are reading the other state through a `terraform_remote_state` data source, and looking up the real resource by name or tag with a normal data source.

```
data "aws_vpc" "main" {
  tags = { Name = "prod-main" }
}

resource "aws_security_group" "app" {
  name   = "shop-app"
  vpc_id = data.aws_vpc.main.id
}
```

Looking up by tag avoids giving every consumer read access to another team's state, which can contain secrets. Publish only the values others need, for example subnet IDs, and name them as a stable interface.

Speed

Several settings help once the structure is sound:

MARGA by EknathaLabs marga.eknathalabs.com

- `terraform plan -refresh=false` skips reading real resources (use only when you know nothing changed outside Terraform).
- `-target` limits a plan to chosen resources, but it is for emergencies, because it can leave things out of step.
- `-parallelism=20` raises the number of concurrent operations (the default is 10), if the provider API allows it.
- Cache providers with `TF_PLUGIN_CACHE_DIR` in the pipeline, so each run does not download them again.

Measure first. A plan that spends most of its time on one slow data source needs a fix there, not more parallelism.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Large configurations: structure, speed and blast... fits together

Part	What it does	What goes wrong
The cost of one big state	Terraform reads every resource in the state during a plan to see if it changed, and each read is an API call.	Giving every stack read access to every state, including secrets.
Split by lifecycle and owner	Put things that change at different speeds, or belong to different teams, in separate stacks, each with its own state.	Creating many tiny stacks with circular dependencies.
Share values between stacks	A stack exposes outputs, and others read them.	Splitting by file name instead of by lifecycle and owner.
Speed	Several settings help once the structure is sound.	Using -target as a normal habit.

Flow: The cost of one big state to Split by lifecycle and owner / Split by lifecycle and owner to Share values between stacks / Share values between stacks to Speed

Incident: Eleven minutes for a tag change

> **Incident:** *A one-line tag change needed an eleven-minute plan, and during that time nobody else could change anything in the account.*

```
$ time terraform plan
...
Plan: 0 to add, 14 to change, 0 to destroy.
real    10m52s
$ terraform state list | wc -l
1812
```

Everything lived in a single state. The team split foundation, network, data and each application into stacks. Plans fell to under a minute, the shop team no longer blocked the data team, and an application stack could not even see the network resources it did not own. They added a short README to each stack describing its outputs as its public interface.

Common mistakes

- Splitting by file name instead of by lifecycle and owner
- Creating many tiny stacks with circular dependencies
- Giving every stack read access to every state, including secrets
- Using -target as a normal habit

Check yourself

Why is a single huge state a problem?

Answer: Plans are slow, the lock serializes everyone, and a mistake has a large blast radius

What is a good boundary between stacks?

Answer: Different change speed or different owners

How can a stack use a value from another team without reading their state?

Answer: Use a normal data source that looks up the real resource by tag or name

When is -target appropriate?

Answer: Rarely, in emergencies

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. Debugging Terraform plans, state and providers

15 minutes. Diagnose failed plans and applies, state locks, drift, unexpected replacements and provider errors using TF_LOG, plan output, state commands and import.

Terraform failures fall into a few families: the configuration is invalid, the state does not match reality, the provider call was rejected by the cloud, or a plan wants to destroy something you did not expect. Knowing the family tells you where to look.

What you will learn

- Reading validate, plan and apply errors
- Turning on detailed logs with `TF_LOG`
- State problems: locks, drift, missing or duplicated resources
- Why a plan shows replacement, and how to prevent it
- Recovering with `import`, `state mv` and `state rm`

Four families of failure

Family	Typical message	First step
Configuration	Unsupported argument, invalid reference	`terraform validate`, `terraform fmt -check`
State	Error acquiring the state lock, resource already exists	`terraform state list`, check the backend
Provider or cloud	403 AccessDenied, quota exceeded, name taken	Read the API error, check permissions and limits
Plan surprise	`must be replaced`, unexpected destroy	Read `# forces replacement` markers

See what the provider sent

```
TF_LOG=DEBUG TF_LOG_PATH=./tf.log terraform plan   # may contain sensitive data, delete after use
terraform plan -out=tfplan && terraform show tfplan
terraform plan -refresh-only                       # show drift without proposing changes
terraform state list
terraform state show aws_instance.web
```

`TF_LOG=DEBUG` shows each API request and response, which turns "Error: 403" into a named permission. Delete the log file afterwards because it can contain secrets.

Plan surprises

```
  # aws_db_instance.main must be replaced
-/+ resource "aws_db_instance" "main" {
      ~ identifier = "shop-db" -> "shop-database" # forces replacement
```

A changed name or other immutable attribute forces replacement. For stateful resources this means data loss. Revert the change, or add `lifecycle { prevent_destroy = true }` to stop an accidental destroy. When you rename a resource in code, use a `moved` block or `terraform state mv` so Terraform treats it as the same object.

Repair the state, carefully

```
terraform import aws_s3_bucket.logs shop-logs-prod     # adopt an existing resource
terraform state mv aws_instance.web aws_instance.app   # rename in state
terraform state rm aws_instance.old                    # forget it without destroying it
terraform force-unlock LOCK_ID                         # only when sure nobody is applying
```

MARGA by EknathaLabs marga.eknathalabs.com

Take a copy of the state first (`terraform state pull > backup.tfstate`), and never edit the state file by hand.

Diagram: Narrowing down a failed Terraform run

Part	What it does	What goes wrong
Validate	terraform validate and fmt catch syntax and reference errors.	Skipping it means slow cloud errors for simple typos.
Read the plan	Look for replacement markers and unexpected destroys.	Applying without reading the plan is how data is lost.
State and lock	state list, show and the backend lock status.	A stale lock or drifted resource blocks or misleads the plan.
Provider and cloud	TF_LOG shows the API call and the exact denial or quota error.	Missing permissions look like random failures without the log.
Fix, plan again	Change code or state, then confirm with a clean plan.	An apply that is not followed by a clean plan hides leftovers.

Flow: Validate to Read the plan / Validate to State and lock / Read the plan to Provider and cloud / State and lock to Provider and cloud / Provider and cloud to Fix, plan again

Incident: the plan that wanted to delete the database

Incident: *A harmless-looking refactor shows* `1 to add, 1 to destroy` *on the production database.*

```
$ terraform plan | grep -B2 -A2 "forces replacement"
  # module.db.aws_db_instance.main must be replaced
      ~ db_subnet_group_name = "shop-a" -> "shop-b" # forces replacement
```

The refactor changed the subnet group name, which RDS cannot change in place. The reviewer caught the destroy line, so the team reverted the name, added `prevent_destroy`, and added a pipeline rule that fails when a plan destroys a tagged production resource.

Common mistakes

- Applying a plan nobody read
- Using `force-unlock` while another run is active
- Hand-editing state
- Leaving `TF_LOG` files that contain secrets

Check yourself

What does "# forces replacement" next to an attribute in a plan mean?

Answer: The attribute cannot change in place, so the resource is destroyed and created again

Which setting reveals the exact API errors a provider received?

Answer: TF_LOG=DEBUG with TF_LOG_PATH

You renamed a resource in code and want to keep the existing object. What do you add?

Answer: A moved block, or run terraform state mv

An apply is interrupted and later runs report a state lock. When is force-unlock acceptable?

Answer: Only after confirming no other run is using the lock

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Terraform

Workflow commands, HCL syntax, state operations, meta-arguments and safe pipeline practices.

Workflow

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform destroy
terraform output
terraform output -raw bucket_name
```

Plan symbol	Meaning
+	Create
~	Update in place
-	Destroy
-/+	Replace (destroy and recreate)

Read the summary line and every `-` or `-/+` before you apply.

Configuration skeleton

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

variable "region" {
  type    = string
  default = "eu-west-1"
}

resource "aws_s3_bucket" "logs" {
  bucket = "acme-logs-${var.environment}"
}

output "bucket_arn" {
  value = aws_s3_bucket.logs.arn
}
```

Variables

```
variable "environment" {
  type        = string
  description = "dev, staging or prod"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Must be dev, staging or prod."
  }
}

variable "db_password" {
  type      = string
  sensitive = true
}
```

Precedence (low to high): default, `TF_VAR_name`, `terraform.tfvars`, `-var-file`, `-var`.

MARGA by EknathaLabs
marga.eknathalabs.com

Expressions

```
locals {
  prefix = "shop-${var.environment}"
  is_prod = var.environment == "prod"
  names   = [for n in var.names : upper(n)]
  size    = local.is_prod ? "m5.large" : "t3.micro"
}
```

Function	Example
`merge(a, b)`	Combine maps
`lookup(map, key, default)`	Safe map access
`cidrsubnet(cidr, 8, 1)`	Carve a subnet
`join(",", list)`	Make a string
`try(expr, fallback)`	Fall back on error

Experiment with `terraform console`.

Meta-arguments

```
resource "aws_subnet" "multi" {
  for_each          = var.subnets          # map or set, stable keys
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value
}

resource "aws_instance" "web" {
  count = var.enabled ? 1 : 0              # on/off switch
  lifecycle {
    prevent_destroy       = true
    create_before_destroy = true
    ignore_changes        = [tags]
  }
}
```

Prefer `for_each` to `count` for collections; `count` shifts indexes when an item is removed.

State

```
terraform state list
terraform state show aws_s3_bucket.logs
terraform state rm aws_s3_bucket.legacy     # forget, do not delete
terraform apply -refresh-only               # record reality without changing it
terraform force-unlock LOCK_ID              # only when no run is active

moved {
  from = aws_s3_bucket.logs
  to   = aws_s3_bucket.audit_logs
}

import {
  to = aws_s3_bucket.legacy
  id = "acme-legacy-reports"
}
```

Remote backend

```
terraform {
  backend "s3" {
    bucket       = "acme-terraform-state"
    key          = "shop/prod/terraform.tfstate"
    region       = "eu-west-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

Enable bucket versioning, restrict access, and keep one state per environment.

MARGA by EknathaLabs
marga.eknathalabs.com

Modules

```
module "site" {
  source  = "../../modules/static-site"
  name    = "marketing-prod"
}

output "site_bucket" {
  value = module.site.bucket_name
}
```

Pin versions of modules you do not own. Run `terraform init` after adding a module.

Pipeline checks

```
terraform fmt -check -recursive
terraform validate
tflint
trivy config .
terraform plan -detailed-exitcode     # 0 none, 1 error, 2 changes
terraform test
```

Workflow with a saved plan

```
terraform fmt -recursive
terraform validate
terraform init -upgrade
terraform plan -out=tfplan -input=false
terraform show tfplan
terraform show -json tfplan | jq '.resource_changes[] | select(.change.actions != ["no-op"]) | {a: \
     .address, c: .change.actions}'
terraform apply tfplan
terraform output -json
terraform providers
terraform version
```

> **Danger:** *Read the plan before you apply it, especially lines marked* `destroy` *or* `must be replaced`*. A change to one argument can force a new resource, such as a database.*

> **Warning:** `terraform apply -auto-approve` *skips the human check. Use it only in a pipeline that applies a reviewed, saved plan.*

State inspection and surgery

```
terraform state list
terraform state show aws_instance.web
terraform state pull > backup.tfstate
terraform state mv aws_instance.web aws_instance.app
terraform state rm aws_instance.old
terraform import aws_s3_bucket.logs acme-logs
terraform force-unlock LOCK_ID
terraform refresh
terraform taint aws_instance.web
terraform apply -replace=aws_instance.web
```

> **Danger:** *Always run* `terraform state pull > backup.tfstate` *before* `state mv`, `state rm` *or* `import`*. A wrong state edit can make Terraform destroy or duplicate real resources.*

> **Danger:** `terraform force-unlock` *is only safe when you are sure that no other run is active. Check who holds the lock first.*

> **Warning:** *Do not edit the state file by hand and do not commit it to a shared repository: it can contain secrets in plain text. Use a remote backend with encryption and locking.*

> **Warning:** `terraform taint` *is deprecated. Use* `apply -replace=ADDRESS`*, which shows the effect in the plan.*

MARGA by EknathaLabs marga.eknathalabs.com

Targeting, variables and workspaces

```
terraform plan -target=module.network
terraform plan -var 'env=staging' -var-file=staging.tfvars
terraform workspace list
terraform workspace new staging
terraform workspace select prod
terraform console
export TF_LOG=INFO TF_LOG_PATH=tf.log
TF_VAR_db_password="$DB_PASSWORD" terraform plan
```

Warning: `-target` *skips dependencies and leaves the state out of step with the code. Use it only for recovery, then run a full plan.*

Warning: `TF_LOG=TRACE` *can write secrets and tokens into the log. Do not attach such logs to tickets.*

Warning: *Check* `terraform workspace show` *before apply. Applying to the wrong workspace is a common cause of production changes.*

Destroy

```
terraform plan -destroy -out=destroy.tfplan
terraform show destroy.tfplan | less
terraform destroy -target=aws_instance.test
lifecycle { prevent_destroy = true }
```

Danger: `terraform destroy` *deletes everything in the state, including databases and buckets. Always review* `plan -destroy`*, set* `prevent_destroy` *on critical resources, and turn on deletion protection in the cloud.*

Warning: *Provider upgrades with* `init -upgrade` *can change behaviour. Pin versions in* `required_providers` *and read the changelog.*

MARGA by EknathaLabs
marga.eknathalabs.com